DERİN ÖĞRENME · KARŞILAŞTIRMALI ANALİZ

S&P 500 Yön Tahmini

Kendi Feedforward Projem ile Samuel Edet'in RNN Çalışmasının Kıyaslanması

BÖLÜM 1

Giriş ve Problem Tanımı

S&P 500 Nedir?

S&P 500, ABD'nin en büyük 500 halka açık şirketinin hisse performansını tek bir rakamda toplayan bir endekstir. Apple, Microsoft, JPMorgan gibi şirketler bu sepetin içindedir. Endeksin yönü, küresel ekonominin genel sağlığını yansıtan en önemli göstergelerden biridir.

Yönünü Tahmin Etmek Neden Zor?

Finansal piyasalar, teknik tabirle **düşük sinyal-gürültü oranına (low signal-to-noise ratio)** sahip sistemlerdir. Bu ne demek? Fiyat hareketleri iki tür bilgiyi içerir:

- **Sinyal:** Fiyatı gerçekten etkileyen ekonomik nedenler (şirket kârları, faiz kararları, arz-talep dengesi).
- **Gürültü:** Milyonlarca yatırımcının psikolojik tepkileri, panik satışları, geçici haber dalgaları.

Finansal verilerde gürültünün oranı sinyalden çok yüksektir. Bu yüzden rastgele tahmin bile yaklaşık %50 doğruluk verir — her 1 puanlık gerçek iyileşme, yatırım stratejisinde ciddi bir avantaj sayılır.

Neden Geleneksel Yöntemler Değil de Derin Öğrenme?

Geleneksel istatistiksel yöntemler (doğrusal regresyon gibi), verinin belli varsayımlara (örneğin değişkenler arasında doğrusal ilişki olması) uyduğunu kabul eder. Ancak finansal piyasalar **doğrusal olmayan (non-linear)** ve **durağan olmayan (non-stationary)** yapıdadır; yani ilişkiler zamanla değişir ve basit kalıplara uymaz.

Derin öğrenme modelleri, bu karmaşık yapıyı çözmek için iki önemli avantaj sunar:

- Verinin altında yatan kalıplar hakkında **önceden varsayım yapmazlar**; doğrudan veriden öğrenirler.
- Milyonlarca ince ayar parametresi sayesinde, **doğrusal olmayan örüntüleri** (nonlinear patterns) yakalayabilirler.

Bu rapor, aynı problemi iki farklı derin öğrenme yaklaşımıyla ele alan iki çalışmayı kıyaslıyor: Benim **Feedforward (Dense) sinir ağımı** ve Samuel Edet'in **Tekrarlayan Sinir Ağları (RNN / LSTM / GRU)** tabanlı akademik çalışmasını. Amaç; her iki yaklaşımın mimari, veri işleme ve sonuç boyutlarında nerede buluşup nerede ayrıldığını net bir şekilde göstermektir.

BÖLÜM 2

İki Farklı Yaklaşım: Modellerin Kıyası

Benim Modelim: Feedforward (İleri Beslemeli) Sinir Ağı

Benim kullandığım **Feedforward Neural Network**, en temel derin öğrenme mimarisidir. Bilgi, girdiden çıktıya **tek yönde** akar. Model, kendisine verilen bir satır veriyi (örneğin bugünkü 9 finansal göstergeyi) işler ve bir tahmin üretir. Ardından bir sonraki satıra geçer — **önceki satırı tamamen unutmuştur.** Hafızası yoktur.

Mimari:

```
Giriş (9 özellik) → Dense(64) → Dropout → Dense(32) → Dropout → Sigmoid
```

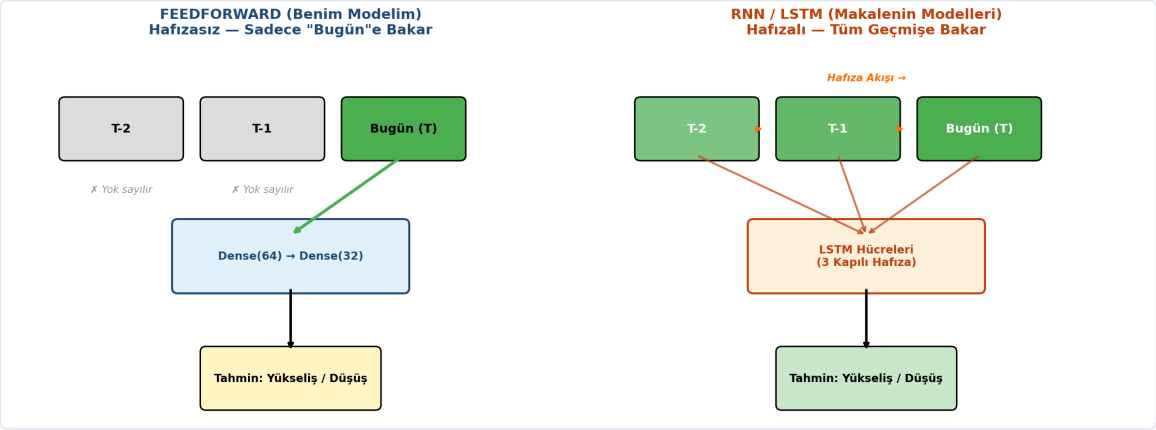
Makalenin Modeli: RNN ve LSTM (Hafızalı Ağlar)

Samuel Edet'in makalesi ise **Tekrarlayan Sinir Ağlarını (Recurrent Neural Networks — RNN)** kullanıyor. Bu ağlar, işledikleri her bilgiyi bir **iç hafızada (hidden state)** saklar ve bir sonraki adımda hem yeni bilgiyi hem de geçmişin özetini birlikte değerlendirir. Böylece zaman içinde biriken örüntüleri yakalayabilirler.

Makalede üç RNN türü inceleniyor:

- **Simple RNN:** En temel hafızalı ağ. Uzun geçmişte eski bilgilerin silikleşmesi ("vanishing gradient") sorunu yaşar.
- **LSTM (Long Short-Term Memory):** Üç kapı (giriş, unutma, çıkış) sayesinde neyi hatırlayıp neyi unutacağına karar verir. Uzun vadeli bağımlılıkları LSTM çok daha iyi yakalar.
- **GRU (Gated Recurrent Unit):** LSTM'in basitleştirilmiş versiyonu; daha az parametreyle benzer iş görür, daha hızlı eğitilir.

> 💡 **METEOROLOG ANALOJİSİ**
>
> **Feedforward**, yarınki havayı tahmin ederken *yalnızca bugünün gazetesine* bakan bir meteorologdur. Dün ne olmuş, hafta boyunca basınç nasıl değişmiş — hiçbirini hatırlamaz. **RNN / LSTM** ise *son bir haftanın tüm gazete arşivini* zihninde tutan bir meteorologdur. Karar verirken hem bugünün verilerini hem de geçmişteki trendleri birlikte değerlendirir.



Şekil 1. *Feedforward yalnızca anlık veriye bakarken, RNN/LSTM geçmiş adımları zincirleme işler.*

Özellik	Feedforward (Benim)	RNN / LSTM / GRU (Makale)
Hafıza	Yok	Var
Girdi yapısı	2D: (Örneklem, Özellik)	3D: (Örneklem, Zaman Adımı, Özellik)
Zaman serisi uygunluğu	Düşük	Yüksek

BÖLÜM 3

Veri ve Eğitim Stratejisi Kıyaslaması

Veri Setleri: İki Farklı Felsefe

Her iki projede de amaç aynı olmasına rağmen, veriye yaklaşım tamamen farklıdır.

Benim Projem — 9 Teknik Gösterge + Büyük Veri:

Yaklaşık **257.000 satırlık** S&P 500 sektör ETF verisi kullandım. Ham fiyat yerine, piyasanın içindeki bilgiyi farklı açılardan özetleyen 9 teknik gösterge ürettim:

- **Log getiri:** Günlük yüzde değişimin standardize edilmiş hali.
- **Volatilite (10 ve 30 günlük):** Fiyatın ne kadar oynadığını ölçer.
- **RSI (Göreceli Güç Endeksi):** Aşırı alım/satım durumunu gösterir.
- **MACD ve bileşenleri:** Trendin yönünü ve gücünü ifade eder.
- **Bollinger Bantları:** Fiyatın istatistiksel olarak "normal" aralığının sınırlarıdır.

Yaklaşımım, **piyasanın kendi içindeki teknik dinamiklerden** bilgi çıkarmak üzerine kurulu.

Makalenin Projesi — 14 Makro Değişken → 3 Değişken:

Samuel Edet, S&P 500'ü dışarıdan etkileyen 14 değişken topluyor: Önceki üç günün S&P 500 kapanışları, 6 ülke borsa endeksi (Hong Kong, Almanya, İngiltere, Fransa, NASDAQ, Dow Jones) ve 5 büyük ABD şirketinin hissesi (Exxon, P&G, GE, Microsoft, Johnson & Johnson).

Ancak 14 değişkenin hepsini kullanmak doğru değil: Birbirleriyle çok benzer hareket edenler var. Örneğin Microsoft ile GE hisseleri neredeyse paralel dalgalanır. Makale, **korelasyon eşiği 0.5** uygulayarak aşırı benzer olanları eler ve 14 değişkenden geriye sadece **3 tanesini** bırakır:

- **SPYt-1** (önceki günün S&P 500 kapanışı)
- **JNJ** (Johnson & Johnson hissesi)
- **IXIC** (NASDAQ endeksi)

Eğitim Güvenliği: Data Leakage'ı Önlemek

Data Leakage (Veri Sızıntısı), modelin eğitim sırasında aslında elinde olmaması gereken bir bilgiye (örneğin gelecekteki bir ipucuna) erişmesi ve bu sayede "kopya çekmesi" durumudur.

> **Projemde kullandığım önlem:** `train_test_split(..., shuffle=False)` — Bu tek parametre, veriyi rastgele karıştırmak yerine zaman sırasına göre ilk %80'i eğitim, son %20'yi test olarak ayırır. Böylece model geçmişle eğitilir, gelecekle test edilir — aynen gerçek hayattaki gibi.

Overfitting'i Önlemek: Dropout

Overfitting (aşırı öğrenme), modelin eğitim verisini ezberleyip yeni verilere genelleme yapamaması durumudur.

> **Projemde kullandığım önlem:** `Dropout(0.5)` — Eğitim sırasında her adımda nöronların %50'sini rastgele "kapatır". Böylece model tek bir nörona aşırı bağımlı hale gelemez; bilgiyi ağ boyunca dağıtmak zorunda kalır, genelleme kapasitesi artar.

BÖLÜM 4

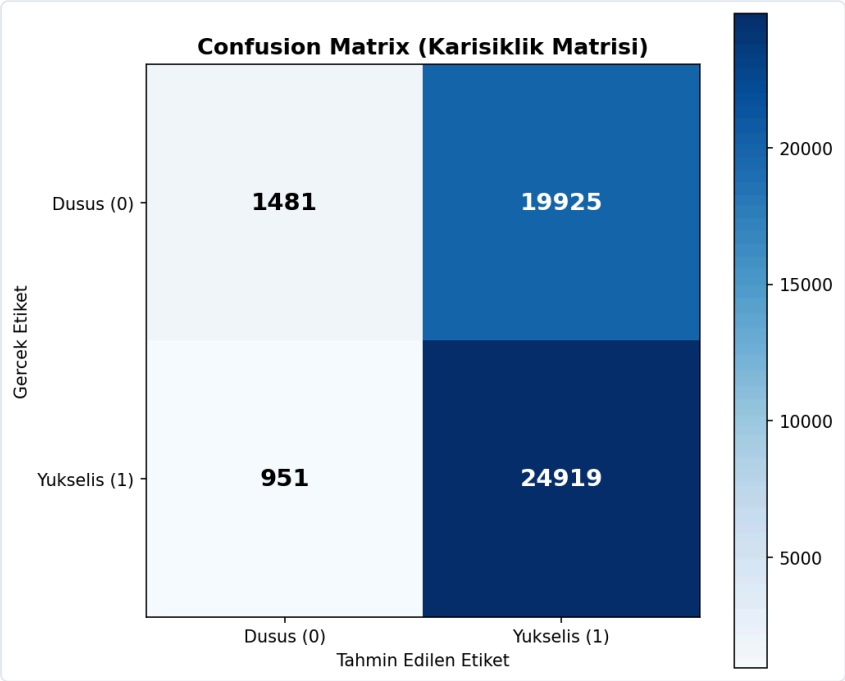
Çarpıcı Sonuçlar: Başarı Uçurumu

İki yaklaşımın sonuçları arasındaki fark, derin öğrenmede **model seçiminin ne kadar kritik olduğunu** çok net gösteriyor.

BENİM MODELİM (FEEDFORWARD)	MAKALENİN MODELİ (SİMPLE RNN)
%55.79	**%75.00**
Test Doğruluğu · AUC: 0.514	LSTM: %74 · GRU: %74

Aradaki fark **yaklaşık 19 puan**. Finansal tahminde bu çok büyük bir fark. Peki bu uçurum neden ortaya çıktı?

Sorun 1: Modelim Sürekli "Yükseliş" Diyor



Şekil 2. *Modelimin tahmin matrisi: Gerçekte düşüş olan 21.406 günün 19.925'inde model yükseliş tahmin etmiş.*

Confusion matrix'e bakıldığında şu net: Modelim neredeyse her örnekte **"Yükseliş (1)"** tahmini yapıyor. Bunun iki nedeni var:

- Eğitim setimdeki yükseliş oranı %54.2 — yani veri kendi başına hafifçe dengesiz.
- Model, gerçek bir ayrıştırma yapamadığı için "çoğunluk sınıfına yasla, kabaca doğru çık" şeklinde bir kestirme yola sapıyor.

> **Çözüm: Class Weight (Sınıf Ağırlığı).** Bu, modele eğitim sırasında "*az rastlanan sınıfı kaçırırsan daha büyük bir ceza alırsın*" mesajını veren bir mekanizmadır. Örneğin veride %54 yükseliş, %46 düşüş varsa, düşüş sınıfına daha büyük bir ağırlık atanır. Böylece model "çoğunluğa yaslanmak" gibi kolay yolu seçemez; her iki sınıfa da eşit dikkat göstermek zorunda kalır. Ben projemde bu mekanizmayı kullanmadım — bir sonraki adımda eklenecek temel iyileştirmelerden biridir.

Sorun 2 (Asıl Mesele): Zamansal Hafıza Eksikliği

Sınıf dengesizliğini düzeltsek bile, %55'ten %75'e sıçramak mümkün olmaz. Çünkü asıl fark, kullanılan modelin **yapısal yeteneğinden** kaynaklanıyor:

> **Benim modelim** bir günü kendi başına değerlendiriyor: "Bugün RSI 45, volatilite düşük. Bu durumda ne olur?" **RNN/LSTM** ise şunu diyor: "Son 5 günde RSI 30'dan 45'e yükseldi, volatilite yavaşça azalıyor, MACD güç kazanıyor. Bu *dizilim* bir yükseliş habercisi." Finansal zaman serilerinde bilginin büyük kısmı, tek bir anda değil, **ardışık değişim dizilerinde** saklıdır. Feedforward bu dizilimi göremez; RNN ve LSTM tam olarak bunun için tasarlanmıştır.

Yani kısaca: **%20'lik devasa başarı farkının asıl kaynağı, hafızasızlıktır.** Finansal zaman serisine hafızasız bir modelle yaklaşmak, bir futbol maçını tek bir fotoğraf karesine bakarak yorumlamaya benzer — birçok önemli bilgi görünmez kalır.

BÖLÜM 5

Sonuç ve Gelecek Adımlar

Bu Projenin Bana Öğrettikleri

Yaklaşık %55'lik bir doğruluk skoru ilk bakışta düşük görünebilir. Ancak asıl değerli olan, bu projenin bana derin öğrenmenin **temel disiplinlerini pratik bir problem üzerinde yaşayarak** öğretmesidir.

Projede doğru yapılanlar:

- ✓ **Leakage-safe pipeline:** `shuffle=False` ve yalnızca eğitim setine uygulanan `StandardScaler` ile zaman sırası ve veri bütünlüğü korundu. Model geçmişle eğitildi, gelecekle test edildi — gerçekçi bir değerlendirme elde edildi.
- ✓ **Overfitting önlemleri:** Dropout (0.5) ve Early Stopping (patience=10) birlikte kullanıldı; model sadece 14 epoch'ta durdu ve ezber yerine genelleme yaptı.
- ✓ **Zengin özellik seti:** Ham fiyat yerine 9 finansal gösterge (volatilite, RSI, MACD, Bollinger vb.) ile piyasanın farklı yönleri modele sunuldu.
- ✓ **Büyük veri hacmi:** 257.000 satırlık örneklem, sonuçların istatistiksel olarak güvenilir olmasını sağladı.

Projede eksik kalanlar:

- ✗ **Hafızasız model mimarisi:** Feedforward, zaman serisinin ardışık yapısını yakalayamıyor. Bu, %20'lik başarı farkının asıl nedeni.
- ✗ **Sınıf dengesizliği yönetilmedi:** `class_weight` kullanılmadığı için model çoğunluk sınıfına (yükseliş) yaslanmaya meyilli hale geldi.
- ✗ **Sistematik mimari seçimi eksikti:** Nöron ve epoch sayıları tek bir konfigürasyonda sabitlendi; makaledeki gibi deneysel tasarımla (DoE) optimize edilmedi.

Baseline'dan Sonraki Adımlar

Bu projenin en önemli katkısı, iyi bir **temel model (baseline)** olmasıdır. Baseline, bir derin öğrenme çalışmasında "bundan daha iyisini yapmam gerek" demek için kullanılan referans noktadır. Projem, gelecekteki daha karmaşık modellerin performansını kıyaslayacağım sağlam bir zemindir.

Bir sonraki somut adımlar şunlardır:

- **Sliding Window (Kayan Pencere) veri yapısı:** Tek bir günün özelliklerini modele vermek yerine, "son 10 günün özelliklerini bir pencere halinde topla, bir sonraki günün yönünü tahmin et" yaklaşımı. Veri yapısı 2D'den **3D'ye (Örneklem × Zaman Adımı × Özellik)** dönüşür. Bu yapı, RNN modellerinin gerekli girdi formatıdır.
- **LSTM mimarisine geçiş:** Sliding Window ile hazırlanan veri, LSTM katmanlarından geçirilir. Bu, makaledeki %74-75 seviyesine ulaşmanın doğrudan yoludur.
- **Class Weight ve eşik ayarı:** Sınıf dengesizliği ile ilgilenmek ve karar eşiğini (0.50 yerine 0.55 gibi) optimize etmek, pratik kullanımda büyük fark yaratır.

Kapanış

Derin öğrenmede başarı, tek bir "doğru model" değil; **probleme uygun mimariyi seçme** ile gelir. Finansal zaman serisi için o doğru mimari Feedforward değil, LSTM'dir. Benim Feedforward modelim, bu gerçeği somut sayılarla bana öğretti — ve bir sonraki seviye için sağlam bir baseline kurdu.

Sonraki adım net: Sliding Window veri yapısı + LSTM mimarisi.